

The End of Software Engineering:

How AI Agents Are Fundamentally Restructuring the Software Paradigm

Zhenfeng Cao

Lingxi Intelligent Investment (Shenzhen) Development Co., Ltd.

info@stellarsea.com

June 5, 2026

Abstract

For over half a century, software engineering has operated on a foundational premise: human engineers decompose problems, encode decision logic into static code, and manually adapt that code as requirements evolve. This paper argues that the emergence of AI agents – systems where large language models serve as the primary reasoning engine, dynamically generating and discarding code as an instrumental resource – constitutes not an incremental improvement but a fundamental restructuring of the software paradigm. Drawing on first-principles analysis of complexity scaling, we formalize the distinction between traditional software (where code is the carrier of decision logic) and agentic systems (where code is ephemeral tooling for an LLM-driven reasoning loop). We trace the historical arc from licensed software to SaaS to what we term Agent-as-a-Service (AaaS), showing that each shift transferred additional complexity away from end-users. We introduce the concept of Agentic Engineering as an emergent discipline – distinct from software engineering in its core object of study, control model, and human role. Through analysis of recent benchmark evidence including SWE-bench Verified, EvoClaw, and LangChain’s multi-agent coordination studies, we demonstrate both the transformative potential of the agentic paradigm and its current limitations. We conclude with a four-stage roadmap toward self-evolving agent ecosystems and concrete recommendations for practitioners navigating this transition.

1 Introduction

Software engineering, as codified at the 1968 NATO Conference [1], was born from a crisis: systems were growing in complexity beyond what ad-hoc programming practices could manage. The discipline’s founding insight was that rigorous methodologies—structured design, modular decomposition, configuration management, systematic testing—could tame this complexity. For five decades, this bet largely paid off. We moved from waterfall to agile, from monoliths to microservices, from manual deployment to CI/CD.

Yet a deeper structural problem persisted. As Brooks observed in *The Mythical Man-Month* [2], software complexity exhibits a fundamentally different scaling behavior than other engineering domains. Unlike bridges or circuits, software has no manufacturing step—the design *is* the product. Every new feature, every edge case, every integration point adds to a combinatorial explosion of possible states and interactions that Brooks characterized as “essential complexity”: complexity inherent to the problem itself, not accidental to the implementation.

This paper contends that the emergence of AI agents does not merely offer a new tool within the existing paradigm. Rather, it dissolves the very premise on which software engineering was founded. When a large language model (LLM) [12] can understand a task, decompose it into subtasks, dynamically generate code to execute those subtasks, and discard that code when it’s no longer needed, the role of code changes from *the system itself* to *an ephemeral instrument of reasoning*. This shift is as fundamental as the transition from analog circuits to stored-program computers.

We make three central claims:

1. **First-Principles Necessity.** The agentic paradigm is not a market preference but an inevitable consequence of complexity scaling laws. Traditional software requires human engineers to explicitly encode every decision; LLM-based agents can navigate complexity non-linearly by outsourcing reasoning to models whose capacity grows with training compute.
2. **Paradigm Shift, Not Optimization.** The transition from “AI → Software → Result” to “Agent → Result” eliminates the software artifact as a necessary intermediary—comparable to how SaaS eliminated on-premise installation as a necessary intermediary. We formalize this as the third major paradigm shift in software delivery.
3. **Emergent Discipline.** Agentic Engineering is emerging as a distinct practice with its own concepts, tools, and metrics. Its practitioners are not “better programmers” but a fundamentally different role: intent architects, agent coordinators, and outcome auditors.

The remainder of this paper is structured as follows. Section 2 presents a first-principles analysis of traditional software and agent-based systems, including a formal complexity argument. Section 3 traces the historical paradigm shifts in software delivery and positions AaaS as the logical endpoint. Section 4 defines Agentic Engineering as a discipline and contrasts it with traditional software engineering. Section 5 reviews empirical evidence from recent benchmarks, acknowledging both breakthroughs and persistent challenges. Section 6 proposes an evolutionary roadmap. Section 7 concludes with implications for practitioners and the research community.

2 First-Principles Analysis

2.1 The Nature of Traditional Software

We begin with a precise definition.

Definition 2.1 (Traditional Software System). *A traditional software system S is a tuple $S = (C, D, E)$ where:*

- C is a set of computational resources (CPU, memory, I/O);
- D is a set of deterministic decision rules encoded in source code;
- E is an execution environment that evaluates D against inputs to produce outputs.

The critical property is that D is static with respect to execution: all decision logic must be explicitly written by human engineers before the system encounters any input.

Under this definition, every feature addition, every bug fix, every adaptation to a changing environment requires a human to (a) understand the change needed, (b) locate the correct position in D , (c) modify the logic without introducing regressions, and (d) verify correctness. The cost of each change is a function of the size of D and the density of its internal dependencies.

2.2 The Complexity Barrier

Brooks [2] distinguished between *accidental complexity* (artifacts of particular implementations) and *essential complexity* (inherent to the problem). While decades of advances—higher-level languages, frameworks, automated testing—have systematically reduced accidental complexity, essential complexity remains unbounded. In fact, as systems grow, the interaction surface between components grows combinatorially.

Proposition 2.1 (Complexity Scaling). *For a system with n components, each potentially interacting with any other, the number of possible interaction paths $P(n)$ is bounded by:*

$$P(n) \in \Theta(2^n) \quad (1)$$

This arises because each of the $\binom{n}{2}$ pairs may or may not have a meaningful interaction, yielding $2^{\binom{n}{2}}$ possible dependency graphs. While real systems do not realize all configurations, the upper bound on complexity grows exponentially, while human cognitive capacity to reason about these interactions is essentially constant.

This mismatch is the deep structural reason why software projects experience declining marginal productivity as they grow. The traditional response—hierarchical decomposition, modular interfaces, encapsulation—reduces the constant factor but does not change the asymptotic behavior.

2.3 Agentic Systems: A Formal Model

In contrast, an agentic system operates on fundamentally different principles.

Definition 2.2 (AI Agent System). *An AI agent system A is a tuple $A = (M, \mathcal{T}, \mathcal{M}, \Pi)$ where:*

- M is a large language model serving as the reasoning engine;
- $\mathcal{T}$ is a set of executable tools (code interpreters, APIs, databases, file systems);
- $\mathcal{M}$ is a memory subsystem (short-term context, long-term vector store);
- Π is a planning mechanism that decomposes user intent into action sequences.

The system operates by iteratively executing: $a_t \leftarrow M(s_t, \mathcal{M})$, $s_{t+1} \leftarrow \text{exec}(a_t)$, where s_t is the system state at time t and a_t is the action chosen by the model.

The key distinction is that in an agentic system, the decision logic is *generated at run-time*. The LLM M can dynamically produce code, invoke tools, and adjust its behavior based on intermediate results—none of which was explicitly pre-programmed. The code it generates is not the system; it is a transient artifact, produced and discarded as needed.

This distinction maps cleanly to Karpathy’s “Software 2.0” framework [3], but extends it further. In Karpathy’s formulation, neural networks replace hand-crafted program logic with learned weights. Agentic systems go a step further: the neural network does not merely *replace* the program—it *writes programs on demand*, using code as a tool in service of broader reasoning goals. This pattern is consistent with the ReAct framework [9], which demonstrated that interleaving reasoning traces with tool-use actions substantially improves task performance, and with Chain-of-Thought prompting [8], which showed that explicit intermediate reasoning steps unlock latent capabilities in LLMs.

2.4 Why Agents Inevitably Scale Better

Consider a task T whose solution requires reasoning over a space of size N . Under the traditional paradigm:

- A human engineer must mentally traverse this space to identify the solution path.
- The path must then be encoded as a static program.
- Human cognitive capacity C_H is essentially fixed.
- Thus, for $N > C_H$, the task is infeasible at any realistic cost.

Under the agentic paradigm:

- The LLM M traverses the space, with effective capacity C_M that scales with model size and training compute.
- The plan Π decomposes T into subproblems, each handled independently.
- Code is generated only for the specific solution path, not for all contingencies.
- As LLM capabilities improve (which they have been, exponentially), C_M grows correspondingly.

Thus, the agentic paradigm decouples solution capacity from human cognitive limits. This is not a 10% improvement; it is a qualitative change in what kinds of problems can be economically addressed.

3 From SaaS to AaaS: The Third Paradigm Shift

3.1 Three Generations of Software Delivery

The history of commercial software can be understood as a progressive transfer of complexity away from the end-user. Table 1 summarizes this trajectory.

Table 1: Three Generations of Software Delivery

Generation	Core nism	Mecha- nism	Complexity Owner	Revenue Model	Exemplars
Software 1.0 (Local)	Code execute	+ data on- premise	End-user (installation, mainte- nance)	License sale	Microsoft, Oracle
Software 2.0 (SaaS)	Code execute	+ data in cloud	Vendor (in- frastructure, updates)	Subscription	Salesforce, AWS
Software 3.0 (AaaS)	Agent operates	au- tonomously in cloud	Agent (un- derstanding, building, running)	Outcome- based	OpenAI, An- thropic

Each transition follows the same pattern: the party best positioned to absorb complexity absorbs it, and the party least positioned to manage it is liberated from it. SaaS liberated businesses from server rooms; AaaS promises to liberate them from the need to specify *how* a result should be produced—they need only specify *what* result they want.

3.2 The Failure of “AI → Software → Result”

The dominant enterprise AI paradigm to date has been *AI-augmented development*: use LLMs to help human engineers write code faster, within the traditional software life-cycle. We denote this as the “AI → Software → Result” pipeline.

This approach has three structural weaknesses:

1. **Bottleneck persistence.** The human engineer remains the critical path for design decisions, architecture, integration testing, and deployment. AI accelerates code generation (a sub-step of implementation) but does not remove the human from any phase.
2. **Complexity ceiling intact.** The final deliverable remains a traditional software system $S = (C, D, E)$. Its complexity still scales with the size of D , and it still requires human understanding for any modification. AI merely made construction of D somewhat faster.
3. **Iteration latency.** Even with AI assistance, any functional change requires traversing the full chain: requirements → design → code → test → deploy. This latency cannot be reduced below human communication and coordination speeds.

3.3 “Agent → Result”: Eliminating the Intermediary

The alternative paradigm eliminates the software artifact as a necessary intermediary:

1. Human articulates intent and constraints to an agent.
2. Agent autonomously plans, executes (generating code as needed), validates, and delivers the result.
3. Human audits the outcome and provides feedback.

In this model, software is not delivered; *outcomes* are delivered. The agent may generate thousands of lines of code, execute database queries, call external APIs, produce visualizations—all ephemerally. What persists is the agent’s *capability*, not its intermediate artifacts. Kumar and Ramagopal [7] capture this distinction precisely: “AI coding agents excel at translating intent into code within a single user-driven session. Agentic engineering operates at a higher level of abstraction—it’s a control plane that orchestrates cross-team workflows, maintains long-term memory across agents, and manages state and traceability across the full software delivery lifecycle.”

4 Agentic Engineering: A New Discipline

4.1 Defining the Field

Agentic Engineering, formally introduced by LangChain in April 2026 [7], is defined as “a multi-agent coordination model where AI agents function as digital team members—each with defined roles, shared memory, and a unified observability layer—to drive software through the entire delivery pipeline, not merely to generate code faster.”

Wang et al. [4] provide a foundational taxonomy of LLM-based agents in software engineering, identifying three core modules; a complementary survey by Guo et al. [13] offers a systematic treatment of multi-agent collaboration patterns and progress in LLM-based multi-agent systems.

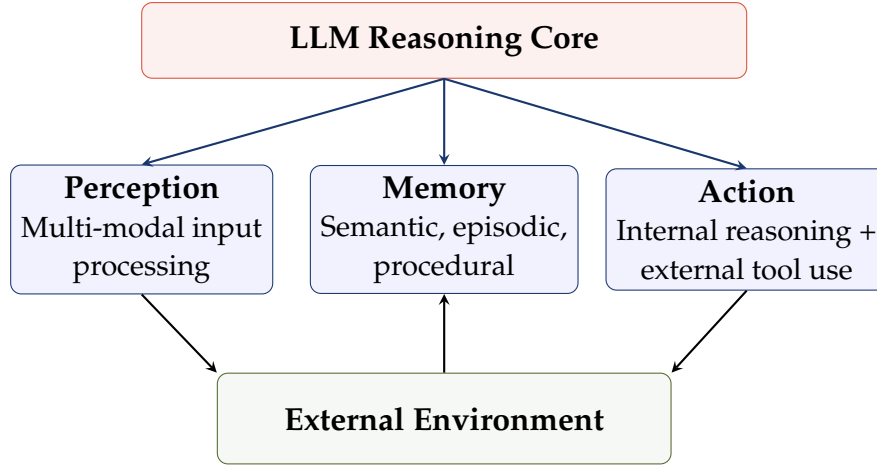


Figure 1: The LLM-based agent framework for software engineering, adapted from Wang et al. [4]. The perception module handles multi-modal input; the memory module maintains semantic, episodic, and procedural knowledge; the action module executes both internal reasoning and external tool invocations. All are orchestrated by the LLM reasoning core.

A concrete realization of this architecture can be observed in Hermes Agent [14], an open-source framework by Nous Research that operationalizes the perception-memory-action model with a distinctive self-evolution mechanism. Its most consequential feature is a closed learning loop: after completing complex tasks, the agent autonomously creates reusable Skills—parameterized procedural modules—that self-improve during subsequent use, automatically patching themselves when found insufficient. Cross-session episodic memory is realized through FTS5-backed conversation search with LLM summarization, enabling the agent to accumulate experiential knowledge over time. The framework’s subagent delegation mechanism further demonstrates early multi-agent coordination in a widely deployed production system.

4.2 Contrasting Agentic and Traditional Engineering

Table 2 maps the key dimensions of difference between the two paradigms.

Table 2: Traditional Software Engineering vs. Agentic Engineering

Dimension	Traditional SE	Agentic Engineering
Core artifact	Source code (static)	Agent system (dynamic)
Control center	Human engineer	LLM reasoning engine
Decision mechanism	Pre-designed logic	Runtime-generated reasoning
Development cycle	Linear (design→code→test)	Autonomous iterative loop
Human role	Code author	Intent architect, coordinator, auditor
Complexity ceiling	Human cognition ($O(1)$)	Model capacity (growing with compute)
Output unit	Functioning software	Delivered outcomes
Error handling	Programmer-defined	Model-adaptive
Evolution	Manual refactoring	Self-modification

4.3 The Human Role Reimagined

Perhaps the most consequential shift is in the human role. In the traditional paradigm, human value was measured by the ability to produce correct, efficient code. In the agentic paradigm, code-generation skill becomes commoditized. The new human differentiators are:

- **Intent articulation.** The ability to specify goals with sufficient clarity and constraint that agents can operate autonomously without producing unintended outcomes.
- **Architectural oversight.** Understanding at the system level how multiple agents should coordinate, what memory should be shared, and where human judgment must intervene.
- **Quality calibration.** Defining what “good” looks like and building evaluation frameworks that agents can use for self-correction.
- **Ethical governance.** Ensuring agent behavior aligns with organizational values, legal requirements, and societal expectations.

We believe the implications for individual practitioners are profound: as agentic capabilities mature, the productivity multiplier for those who master agent orchestration will far exceed the traditional “10x engineer” benchmark—not through faster typing, but through the ability to coordinate swarms of agents toward complex outcomes. The ceiling is not fixed; it rises with each advance in model capability and orchestration infrastructure.

5 Empirical Evidence and Current Limitations

5.1 Breakthrough Results

The empirical record provides strong evidence for the agentic thesis. We highlight four representative data points.

SWE-bench Verified. Ma et al. [5] demonstrated that Lingma SWE-GPT 72B, an open development-process-centric model, resolves 30.20% of GitHub issues on SWE-bench Verified—approaching GPT-4o’s 31.80% while being fully open. Notably, even the 7B variant resolved 18.20%, proving that small models can perform meaningful automated software engineering when trained on process data rather than static code alone. This represents a 22.76% relative improvement over Llama 3.1 405B, a model nearly 6× larger.

Multi-Agent Coordination. Kumar and Ramagopal [7] report results from a pilot study deploying coordinated agent swarms across 20+ enterprise debugging workflows. The coordinated agent system reduced root-cause identification time by 93%, saving over 200 engineering hours in a single month. Critically, these gains came not from better individual agents but from *orchestration*—the ability to maintain shared context across agents, to parallelize investigation, and to cross-validate findings.

Self-Evolution. Hermes Agent [14], an open-source framework by Nous Research with over 179,000 GitHub stars, provides the most complete realization of the self-evolution principle in a production system. Its architecture implements a closed learning loop: after completing complex tasks, the agent autonomously creates reusable “Skills”—parameterized procedural modules that capture successful strategies. Critically, these skills self-improve during use—when a skill is invoked and found lacking, the agent patches it automatically, accumulating refinements over successive interactions. This pattern—create, use, detect weakness, self-patch—operates without human intervention, embodying precisely the self-evolution dynamic that distinguishes agentic systems from traditional software. Cross-session continuity is maintained through FTS5-backed conversation search with LLM summarization, enabling the agent to recall and build upon prior experiences. The framework’s subagent delegation mechanism further demonstrates early multi-agent coordination in a widely deployed system.

Generalization. Wang et al. [4] catalog hundreds of studies applying LLM-based agents across the full software lifecycle: requirements analysis, architecture design, code generation, testing, debugging, deployment, and maintenance. The breadth of coverage suggests that the agentic pattern is not limited to narrow tasks but generalizes across software engineering activities.

5.2 Persistent Challenges

Despite rapid progress, significant challenges remain. The EvoClaw benchmark [6] provides the most sobering data. Deng et al. constructed a benchmark requiring agents to perform *continuous* software evolution—not isolated issue fixes but sustained development across commit histories, where each change must preserve system integrity and where errors accumulate. Their key finding:

“Overall performance scores drop significantly from > 80% on isolated tasks to at

most 38% in continuous settings, exposing agents' profound struggle with long-term maintenance and error propagation." [6]

This reveals four core challenges:

1. **Context drift.** As codebases grow beyond the effective context window, agents lose coherent understanding of system-wide invariants and dependencies.
2. **Error propagation.** A small error in an early commit cascades into compounding failures in subsequent work, and agents lack robust mechanisms for detecting and recovering from these chains.
3. **Technical debt awareness.** Agents do not currently model the long-term costs of their design decisions—they optimize for immediate task completion without considering maintainability.
4. **Verification fidelity.** Automated testing remains incomplete; agents can pass tests while introducing subtle semantic errors that only manifest under novel inputs.

Figure 2 visualizes the performance cliff that EvoClaw reveals.

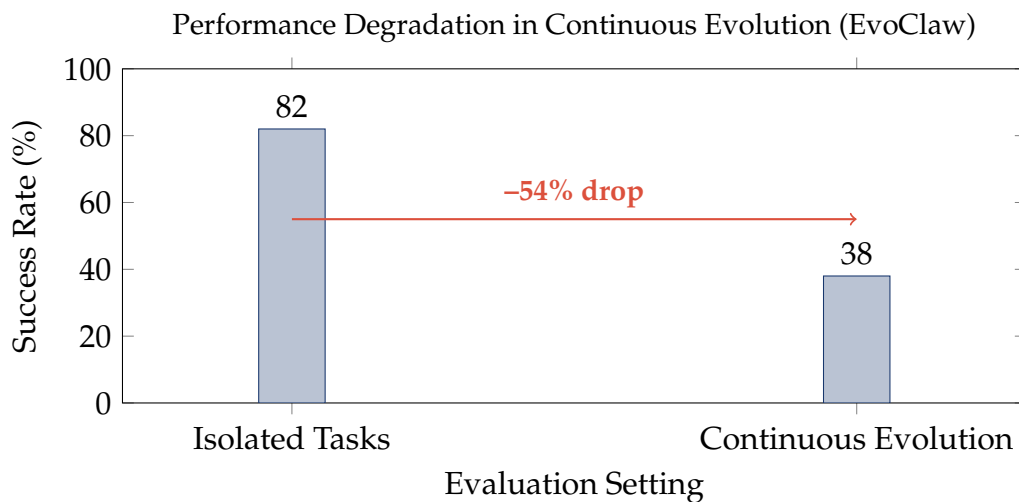


Figure 2: Agent performance on the EvoClaw benchmark [6]. When evaluated on continuous software evolution (requiring sustained development across commits with error accumulation), success rates collapse from over 80% to at most 38%. Data based on evaluation of 12 frontier models across 4 agent frameworks.

5.3 The Gap Analysis

The gap between isolated-task performance ($> 80\%$) and continuous-evolution performance ($< 38\%$) quantifies the distance between current agent capability and the threshold for fully autonomous software engineering. This gap is not fundamental—it reflects limitations in context management, memory architecture, and verification mechanisms that are active areas of research. But it serves as an important calibration: agentic engineering is real and transformative today as an *augmentation* paradigm, but will require several more years of concentrated research before fully autonomous software development becomes reliable in production settings.

6 Evolutionary Roadmap

Based on current capabilities and trajectories, we propose a four-stage roadmap for the evolution of agentic engineering. Table 3 summarizes.

Table 3: Four-Stage Evolution of Agentic Engineering

Stage	Agent Capability	Capabilities	Key Technologies	Technologies	Human Role	Representative Systems
I. Tool-Augmented	Code completion, single-issue fixes, simple script generation		In-context learning, RAG		Author + reviewer	GitHub Copilot, Claude Code
II. Single-Task Autonomous	End-to-end feature building, debugging, system maintenance		Planning tool use, self-correction	+ self-	Intent architect + auditor	Devin, OpenHands
III. Multi-Agent Teams	Coordinated swarms for large systems, full-lifecycle management		Shared memory, role specialization, orchestration		PM + architect + auditor	LangChain orchestration, MetaGPT [11]
IV. Self-Evolving Ecosystems	Autonomous discovery, learning, reproduction, adaptation		Meta-learning, self-modification, ecosystem governance		Goal setter + ethics governor	AGI assistants (prospective)

6.1 Stage I: Tool-Augmented (2023–2025)

The current dominant mode. Agents serve as assistants within human-led workflows. The breakthrough has been in coding: models can generate, explain, and debug code at near-expert level for well-scoped tasks. The limitation is that the human must still decompose problems, design architecture, and verify correctness.

6.2 Stage II: Single-Task Autonomous (2025–2027)

Agents begin to own complete tasks from specification to deployment. Systems like Devin and OpenHands demonstrate that agents can autonomously navigate codebases, implement features, and submit pull requests. The human shifts from “doing” to “specifying what to do and verifying what was done.”

6.3 Stage III: Multi-Agent Teams (2026–2029)

Specialized agents coordinate as teams, mirroring human engineering organizations. A “product manager agent” translates business requirements into technical specifica-

tions; “architect agents” design system structure; “developer agents” implement components; “QA agents” test and validate. Shared memory and observability become critical infrastructure. The LangChain pilot [7] represents an early validation of this pattern.

6.4 Stage IV: Self-Evolving Ecosystems (2028+)

Agents gain the ability to improve their own architectures, spawn specialized sub-agents for new problem domains, and adapt to environmental changes without human intervention. At this stage, the distinction between “software” and “agent” dissolves entirely—the agent *is* the system, and it evolves continuously. Human involvement shifts to meta-level governance: setting ethical boundaries, defining value functions, and ensuring alignment.

7 Implications and Recommendations

7.1 For Practitioners

The transition to agentic engineering demands a deliberate re-skilling strategy:

1. **Shift from code production to intent engineering.** The most valuable skill is no longer writing code efficiently but articulating tasks with sufficient clarity, context, and constraints that agents can execute them correctly.
2. **Build agent orchestration competence.** Understanding how to decompose work across agents, manage shared memory, and design evaluation rubrics will differentiate effective practitioners.
3. **Invest in observability infrastructure.** Agent systems require fundamentally different monitoring than traditional software. Tracing an agent’s reasoning chain, detecting hallucinations, and measuring outcome quality demand new tooling.
4. **Adopt a “human-in-the-loop, agent-in-the-driver’s-seat” posture.** The most effective model today is neither fully autonomous nor fully human-driven. Agents should own execution; humans should own intent, critical judgment, and ethical oversight.

7.2 For Researchers

Several open problems emerge with particular urgency:

1. **Long-context state management.** As EvoClaw demonstrates, agents lose coherence over extended development sequences. Architectures for compressing, indexing, and retrieving relevant context at scale are critical.
2. **Verification in open-ended settings.** Current benchmarks test isolated correctness; real-world systems require guarantees of safety, reliability, and maintainability over time. New verification frameworks that capture these temporal dimensions are needed.

3. **Agent alignment at scale.** As agents become more autonomous and are composed into teams, ensuring that their collective behavior aligns with human values becomes both more important and more difficult.
4. **Economic models.** How should agentic services be priced? Outcome-based pricing (per resolved issue, per deployed feature) may replace subscription and usage-based models, but the incentive structures and risk allocation need careful analysis.

7.3 For Organizations

Organizations should begin preparing for the agentic transition now:

1. **Identify agent-ready workflows.** Not all software work is equally amenable to agent automation. Tasks with clear success criteria, well-defined scope, and existing test infrastructure are ideal starting points.
2. **Invest in evaluation frameworks.** The quality of agent output depends critically on the quality of the evaluation signal. Organizations should build test suites that go beyond correctness to measure robustness, maintainability, and alignment with business intent.
3. **Redesign team structures.** As individual productivity multiplies through agent leverage, team topologies must evolve. Smaller teams of “agent orchestrators” may replace larger teams of developers, with corresponding shifts in hiring, promotion, and career development.

8 Conclusion

This paper has argued that the emergence of AI agents constitutes a paradigm shift in software, not a tool upgrade. The transition from “AI → Software → Result” to “Agent → Result” eliminates the static software artifact as a necessary intermediary, just as SaaS eliminated on-premise installation and cloud eliminated physical infrastructure before it.

The shift is grounded in first principles. Traditional software requires human engineers to encode all decision logic explicitly; the complexity of this task grows exponentially with system size while human capacity remains fixed. Agentic systems outsource decision-making to LLMs whose capacity scales with training compute, decoupling solution capability from human cognitive limits. This is a qualitative change in what kinds of software problems become economically tractable.

Yet we are still in the early stages. Benchmarks like EvoClaw reveal a stark gap between isolated-task performance and sustained autonomous development. The current moment calls for ambitious but calibrated investment: embrace agentic engineering as the dominant paradigm for augmentation while recognizing that fully autonomous software engineering remains a multi-year research challenge.

Agentic Engineering is emerging as a distinct discipline with its own concepts, tools, and professional identity. Its practitioners will not be programmers who learned new tools but a new kind of professional: intent architects who direct swarms of AI agents toward complex outcomes. The old software engineering is ending; the new one has already begun.

Acknowledgments

The author thanks the open-source community for making research artifacts and benchmarks publicly available, and the teams behind SWE-bench, EvoClaw, and LangChain for their foundational contributions to agent evaluation infrastructure.

References

- [1] P. Naur and B. Randell, Eds., *Software Engineering: Report on a Conference Sponsored by the NATO Science Committee*. Garmisch, Germany: NATO, 1968.
- [2] F. P. Brooks, *The Mythical Man-Month: Essays on Software Engineering*. Reading, MA: Addison-Wesley, 1975. (Anniversary Edition with new chapters, 1995.)
- [3] A. Karpathy, “Software 2.0,” *Medium*, Nov. 2017. [Online]. Available: <https://karpathy.medium.com/software-2-0-a64152b37c35> (Accessed: June 4, 2026).
- [4] Y. Wang, W. Zhong, Y. Huang, E. Shi, M. Yang, J. Chen, H. Li, Y. Ma, Q. Wang, and Z. Zheng, “Agents in Software Engineering: Survey, Landscape, and Vision,” *arXiv preprint arXiv:2409.09030*, 2024.
- [5] Y. Ma, R. Cao, Y. Cao, Y. Zhang, J. Chen, Y. Liu, Y. Liu, B. Li, F. Huang, and Y. Li, “Lingma SWE-GPT: An Open Development-Process-Centric Language Model for Automated Software Improvement,” *arXiv preprint arXiv:2411.00622*, 2024.
- [6] G. Deng, Z. Chen, Z. Yu, H. Fan, Y. Liu, Y. Yang, D. Parikh, R. Kannan, L. Cong, M. Wang, Q. Zhang, V. Prasanna, X. Tang, and X. Wang, “EvoClaw: Evaluating AI Agents on Continuous Software Evolution,” *arXiv preprint arXiv:2603.13428*, 2026.
- [7] R. Kumar and P. Ramagopal, “Agentic Engineering: How Swarms of AI Agents Are Redefining Software Engineering,” *LangChain Blog*, Apr. 2026. [Online]. Available: <https://www.langchain.com/blog/agentic-engineering-redefining-software-engineering> (Accessed: June 4, 2026).
- [8] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. Le, and D. Zhou, “Chain-of-Thought Prompting Elicits Reasoning in Large Language Models,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- [9] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, “ReAct: Synergizing Reasoning and Acting in Language Models,” in *International Conference on Learning Representations (ICLR)*, 2023.
- [10] X. Wang, Y. Wang, Y. Wan, F. Mi, Y. Li, P. Zhou, L. Shang, X. Jiang, and Q. Liu, “SWE-bench: Can Language Models Resolve Real-World GitHub Issues?” in *International Conference on Learning Representations (ICLR)*, 2024.
- [11] S. Hong, X. Zheng, J. Chen, Y. Cheng, J. Wang, C. Zhang, Z. Wang, S. K. S. Yau, Z. Lin, L. Zhou *et al.*, “MetaGPT: Meta Programming for a Multi-Agent Collaborative Framework,” in *International Conference on Learning Representations (ICLR)*, 2024.

- [12] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell *et al.*, “Language Models are Few-Shot Learners,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- [13] T. Guo, X. Chen, Y. Wang, R. Chang, S. Pei, N. V. Chawla, O. Wiest, and X. Zhang, “Large Language Model based Multi-Agents: A Survey of Progress and Challenges,” in *International Joint Conference on Artificial Intelligence (IJCAI)*, 2024. [Online]. Available: <https://arxiv.org/abs/2402.01680>
- [14] Nous Research, “Hermes Agent: The Self-Improving AI Agent,” 2025–2026. [Online]. Available: <https://github.com/NousResearch/hermes-agent> — Documentation: <https://hermes-agent.nousresearch.com/docs> (Accessed: June 4, 2026).